ML oblig 2: Egen opplært ML modell

Adrian, 30.10/26

1.0 Omfang

Dette prosjektet tar for seg en maskin lært modell som kategoriserer kundeanmeldelser på produkter som enten negative eller positive. Prosjektet er finalisert som en nettside som tar inn brukerinput i form av en produktomtale og gir et ML-basert resultat i retur som positiv eller negativ omtale.

Dette prosjektet var ikke valgt på bakgrunn av å nå suksess ved lansering av ML-modellen, eller som et kommersielt produkt som kan selges som et verktøy til ulike hensiktsmessige fordeler og årsaker. Men det er tenkt at bedrifter eller influencere som ønsker å samle produktanmeldelser effektivt for å forbedre sine varer kan benytte seg av en slik løsning. Imidlertid har de fleste leverandører og selskaper et tilbakemeldingsystem hvor kjøpere anmelder tjenester/ service / varer via kommentar og rangering i form av stjerner eller terningkast, men i tilfellene hvor omtalene skrives utenfor disse, som f.eks i sosiale medier er det vanskeligere å kategorisere omtalene. Sosiale medier er en stor plattform som i mange tilfeller brukes til kommersielle intensjoner, og med kommentarfelt tilgjengelig kan influencere eller selskaper som benytter disse plattformene også innhente omtaler her. Med denne ML løsningen kan man effektivisere arbeidsmengde på å analysere og hente kundeomtaler, ved å mate inn kommentarene eller repost eller spesifikke innlegg tekst som omtaler produktene som input i ML løsningen og få statistikk på positive og negative.

Per nå er ML løsningen begrenset til å kun ta imot en kommentar og gjengi om den er positiv eller negativ basert på hva ML løsningen er trent opp til å kunne klare, f.eks noen ord er mer diskret og ikke så enkel å klassifisere som verken positiv eller negativ, men med utvidelse av ML løsningen kan man se for seg at den skal samtidig også gi sammendrag til brukeren som undersøker anmeldelsene sine. Vi kan vel si at det er potensialet i applikasjonen, men for nå var denne ML modellen valgt primært for at vi skulle ha et prosjekt til å hovedsakelig lære oss selv gjennom selvstudie hvordan ML modellen kan bygges og hvilke metrikker passer oss i ulike tilfeller når vi driver med ML utvikling.

2.0 Metrikker

Uten metrikker ville det ha vært umulig å vite om ML-modellen har lært noe nyttig under trening. Metrikkene man velger skal avgjøres på bakgrunn av problemstillingen modellen skal løse.

Som måleverktøy i ML-modellen utbygd i dette prosjektet har vi dermed valgt accuracy (treffsikkerhet) som hovedmetrikken. Accuracy kalkuleres etter hvor mange rette prediksjoner modellen oppfylte, og delt på hvor mange prediksjoner modellen utførte. Denne målingen treffer godt til vår binære klassifisering (positiv eller negativ) fordi datasettet vårt har likt antall anmeldelser i hver klasse, (likt antall positive som negative). Samtidig bidrar klassebalansen til å holde treffsikkerheten i større grad reelt. Hadde vi hatt en stor klasseubalanse ville det ha ført til at en modell får for høy accuracy ved å nesten alltid velge den klassen som forekommer oftest.

Videre bruker vi også metrikkene precision, recall og F1-score for hver av de to klassene (positiv & negativ). Til slutt bruker vi en forvekslingsmatrise for å se antallet riktige og feilaktige klassifiseringer i hver klasse.

Når det kommer til prosjektets business metric, går det i utgangspunkt på business metrikken er tidsbesparelse ved sortering av produktomtaler som positive eller negative. Dersom vi hadde utarbeidet denne ML løsningen for lansering hadde vi satt et minimumsmål på at brukerne skal kunne samle kommentarene ut ifra negativ eller positiv omtale minst 45% raskere med ML-løsningen enn manuell lese gjennom alle og tolke selv.

3.0 Data

ML-modellen i dette prosjektet lærer av et datasett som består av korte engelske produktanmeldelser (hovedsakelig ladere, mobiltilbehør o.l). Datasettet er hentet fra Amazon-delen av datasettet Sentiment Labelled Sentences. Datasettet inneholder 1000 omtaler fordelt, likt på positive og negative. Altså, dataene er tekst og labelsene er “positiv” og "negative".

ML-modellen i dette prosjektet er trent opp gjennom konseptet Supervised Learning. Dette betyr at modellen trenes opp ved hjelp av annoterte datasett, og ved hjelp av det så skal modellen vår i etterkant kunne annotere fremtidige data som kommer inn som input, korrekt. En enkel analogi: predikere rett på bakgrunn av treningen sin som var å gi den et eksempel (data) med fasit (labels). Dette har vi som nevnt i prosjektet vårt. ML-modellen ble trent opp på eksempler der riktig klasse (positiv & negativ) var allerede kjent. Når det er sagt, så må vi sjekke om labelsene (fasiten) i datasettet er pålitelig slik at modellen ikke lærer av feilmerket eksempler. Vi sikrer oss dette ved rensing - kontrollere at ingen labels mangler, at alle har gyldige verdier, samt at identiske anmeldelser ikke har motstridende labels.

En bekymring gjeldende opptrening av ML-modeller gjennom datasett er eventuelle personvernhensyn og etiske betraktninger. Så i prosjektet kan et mulig personvernhensyn være at omtalene kan inneholde personopplysninger. Slike opplysninger er ikke nødvendig i treningsmaterialet og bør fjernes. Det er samtidig forbehold en etisk begrensning i modellen, den kan feiltolke anmelder ved sarkasme eller blandede meninger.

Et spørsmål vi må alltid stille oss i ML-modeller er, hvordan dataene skal representeres for modellen I prosjektet er omtalene presentert som numeriske vektorer gjennom TF-IDF. Den måler ord etter hvor ofte de forekommer. Sammen med vår ML-modell (logistisk regresjon) lærer den sammenhengen mellom tallene og lablene. Eksempelvis, dersom “Horrible” ofte forekommer i negative anmeldelser, så vil modellen lære at ordet “Horrible” indikerer negativ label. Dette er vår feature engineering.

4.0 Modellering

Vi har utforsket logistisk regresjon som modell for å avgjøre anmeldelsene som negative eller positive. Vi har videre brukt TF-IDF. Vi har forøvrig ikke sammenlignet forskjellige modeller. Derimot undersøkte vi ulike innstillinger for samme modelltype, fremfor å sammenligne flere modelltyper. Det gjorde vi gjennom GridSearchCV med kryssvalidering på treningsdatene.

Etter datarensingen satt vi igjen med 987 anmeldelser (vi fjernet duplikater o.l), inndelt i 198 testanmeldelser og 789 treningsanmeldelser. Denne inndelingen var starifisert med femfold stratifisert kryssvalidering på treningstadene.

Regulariseringsparameten vår “C” tildelte vi med verdiene 0,1 og 1 og 10. Vi forstod at lavere “C” gir sterkere regularisering. Samtidig sammenlignet vi også TF-IDF med “enkeltord” og “ordpar”, fordi med “ordpar” så var det enklere å lære monstre i uttrykk som “not great”.

Baseline ble evaluert mot fasiten i det samme testsettet. Vi fikk 50,5% accuracy. Vi brukte en enkel løsning som alltid predikere den vanligste klassen i treningsdatene.

Når vi ser på modellen som brukte enkeltord og C= 10, fikk vi 80,3% på testsettet. Mens den endelige modellen som var i kombinasjon av enkeltord og ordpar samt C = 10 fikk lavere accuracy på testsettet (77,8%) derimot ble den valgt av kryssvalideringen. Tilsvarer 154 riktige og 44 feil, vi hadde 198 anmeldelser.

For den endelige modellen sitter vi igjen med følge verdier for metrikkene vi valgte

Label	*F1-score*	*Precision*	*Recall*	*Antall testanmeldelser*
Positiv	*0,78*	*0,77*	*0,79*	*98*
Negativ	*0,78*	*0,79*	*0,77*	*100*

Tabell 4.1 Modellering: Metrikker (Recall, Precision, F1-score)

Forvekslingsmatrisen påpeker hvilke feil modellen gjorde.

Faktisk Label	Predikert negativ	Predikert positiv
Negativ	77	23
Positiv	21	77

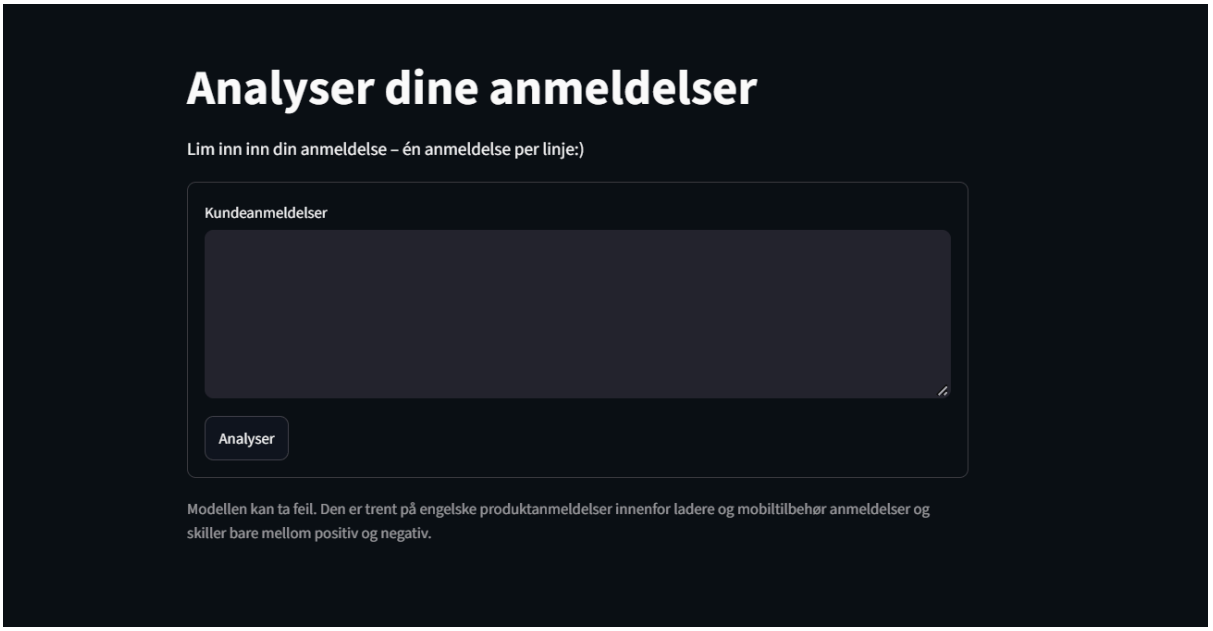
Tabell 4.2 Modellering: Forvekslingsmatriser

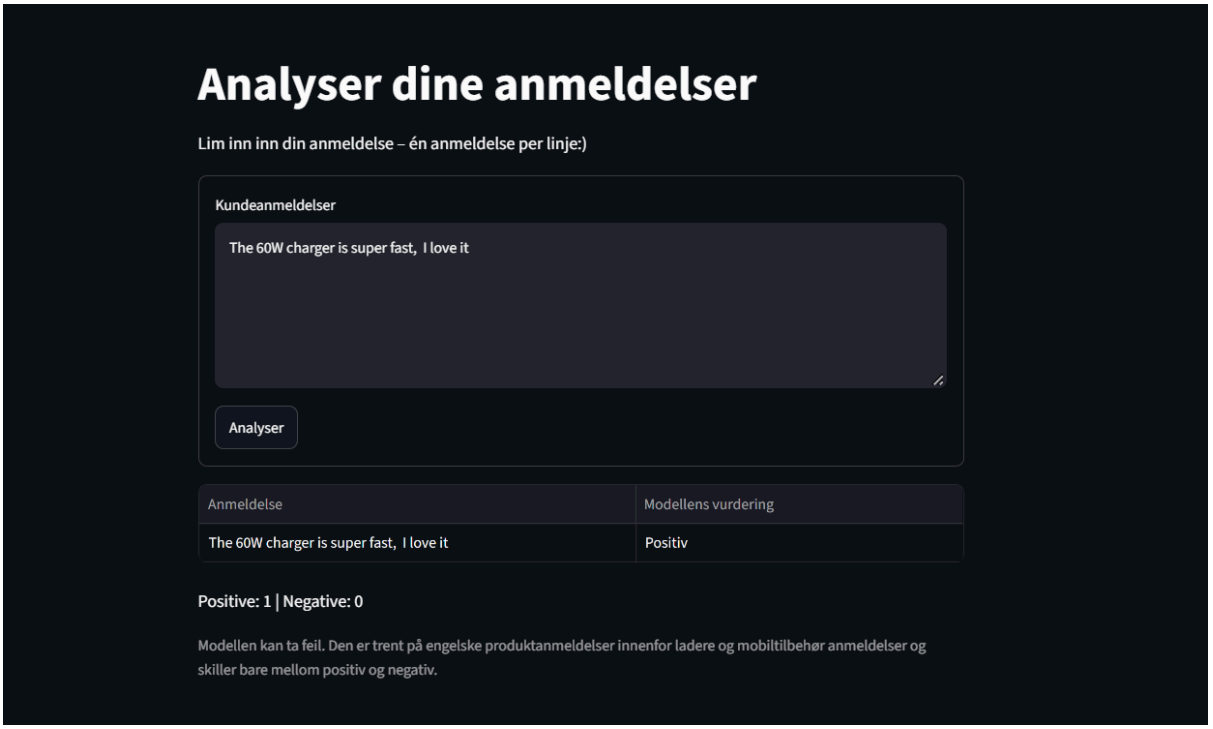
Her ser vi tydelig at ML-modellen predikerte 23 negative anmeldser og antok at 21 positive anmelder som negative. Vi konkluderer dermed med at resultatene trenger å dobbeltsjekkes og kontrolleres av noen.

Forøvrig undersøkte vi feilpredikasjoner ved å skrive egne setninger på nettsiden. Ettersom datasettet var hovedsakelig omtaler og anmeldelser av ladere og mobiltilbehør. Vi prøvde følgende. “The charger stopped working after one day”, dette ble stemplet som positiv, men fikk negativ klasse etter at vi oppdaterte modellen med ordpar identifikasjon. Videre prøvde vi “The train is slow and dirty”, dette ble også stemplet som positivt. Togeksempelet illustrerer at ML-løsningen er begrenset når vi bruker ML-modellen på et bredere kategoriområde, altså produkter bortsett fra mobiltilbehør og ladere o.l. Videre må det også nevnes at det heller ikke ble gjennomført en systematisk analyse av feature importance.

5.0 Deployment

ML-modellen er oppsatt lokalt gjennom en nettside laget med Streamlit. Prediksjonene skal brukes som støtte til sortering av kundeanmeldelser. Vedlikehold vil innebære at ML-modellen trenes opp på nye kvalitetsikre datasett, samt oppdatere nødvendige programpakker som er importert i ML-modellen.





6.0 REFERANSER

[Kotzias, D. (2015)]. *Sentiment Labelled Sentences*. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences

[Scikit-learn]. *TfidfVectorizer*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[Scikit-learn]. *GridSearchCV*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[Streamlit]. *st.form*. Dokumentasjon av skjemaet som brukes til å sende inn anmeldelser på nettsiden. https://docs.streamlit.io/develop/api-reference/execution-flow/st.form

[OpenAI (2026)]. *ChatGPT.* Brukt som støtte til idéutvikling, feilsøking, faglige forklaringer og forslag til rapporttekst. Koden ble kjørt lokalt, og resultatene ble hentet fra programutskriftene og utprøving av nettsiden. http://chatgpt.com